1) Label each of the following scenarios with one of the worst-case asymptotic runtimes shown below (note that you can select the same runtime multiple times):

$$O(1) \qquad O(\log n) \qquad O(n) \qquad O(n \log n) \qquad O(n^2) \qquad O(n^3)$$

(a) Dequeueing from a queue implemented with an array

$O(1)$

(b) Inserting an element into a binary search tree

$O(n)$

(c) Finding the maximum element in a balanced binary search tree

$O(\log n)$

(d) Finding the maximum element in a min-heap

$O(n)$

(e) Performing a single in-place partition step on an array of n elements in Quicksort

$O(n)$

(f) Sorting n elements initially reverse order using Insertion Sort

$O(n^2)$

(g) Sorting n elements initially in sorted order using Selection Sort

$O(n^2)$

2) For each of the following statements, answer whether the statement is **true** or **false** and justify your answer.

(a) If the input sequence is in sorted order, Merge Sort will take $O(n)$ time.

False. Merge sort always takes $O(n \log n)$ time regardless of the input sequence.

(b) An implementation of a min-heap can have $o(\log n)$ time insert and $o(\log n)$ time remove min.

False.

If this were possible, consider performing heap sort.

We perform n insertions, taking $o(n \log n)$ time.

Then we perform n remove min, taking $o(n \log n)$ time.

This gives us an $o(n \log n)$ time comparison based sorting algorithm, which is impossible since we know that sorting algorithms must take $\Omega(n \log n)$ time.

(c) We can modify Radix Sort by using Selection Sort as the intermediate sorting algorithm (to sort the elements by each digit) instead of Bucket Sort.

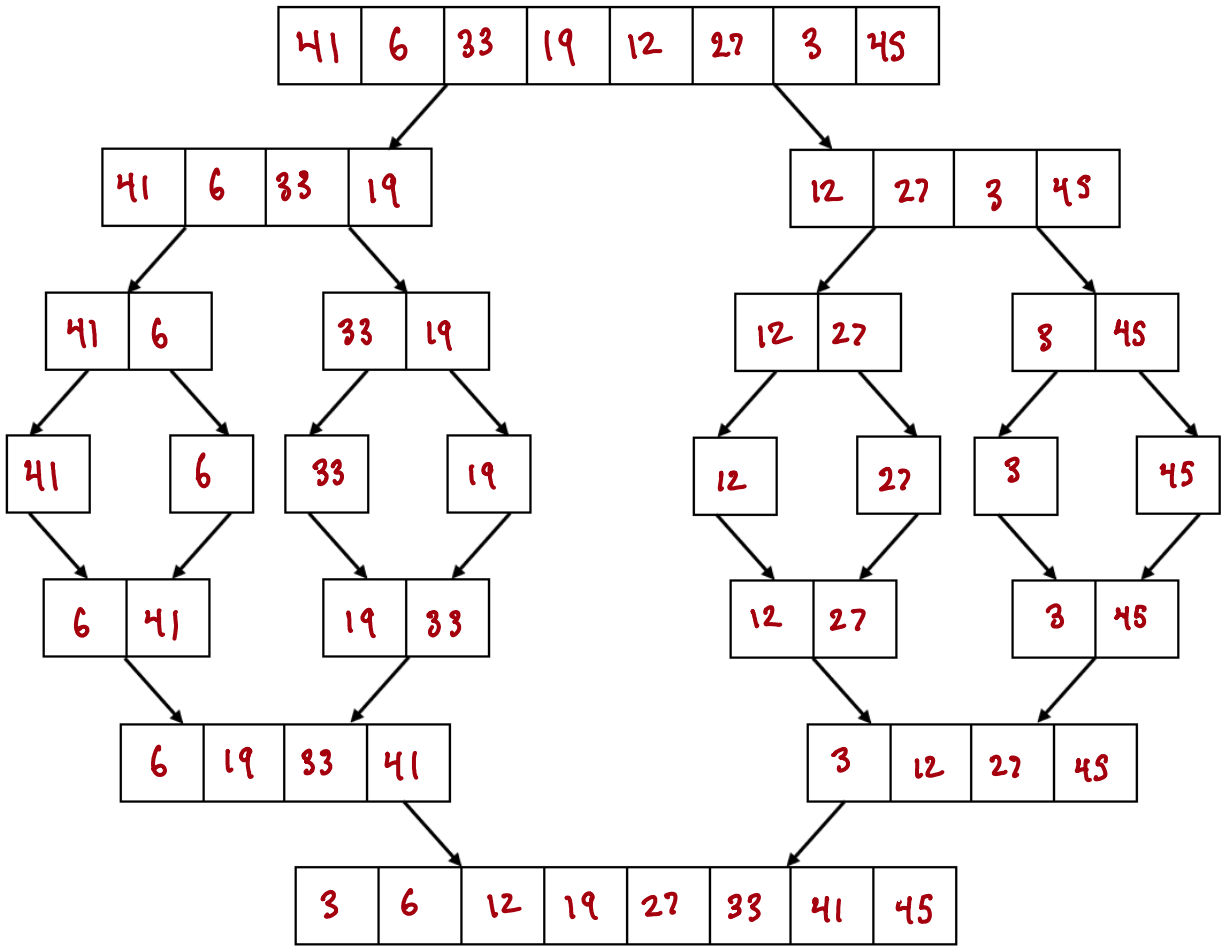
False. The intermediate sorting algorithm we use with Radix sort must be a stable algorithm, otherwise the "progress" of sorting by each digit will be lost.

(d) After inserting a node into an AVL tree, we may have to perform $\Theta(\log n)$ rotations to rebalanced the tree.

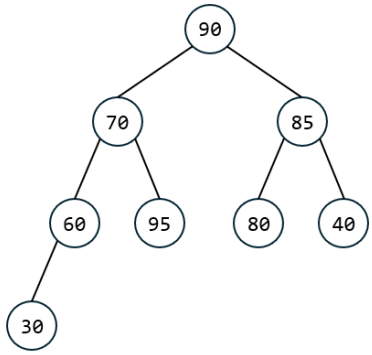
False.

We may need to check $\Theta(\log n)$ nodes to find the root of an imbalance, but we will always perform 1 or 2 rotations after an insertion.

3) Consider running Merge Sort on the following array $A = [41, 6, 33, 19, 12, 27, 3, 45]$. Complete the following diagram which illustrates the levels of recursive calls until arrays of size 1 are reached, then illustrates each merge step.



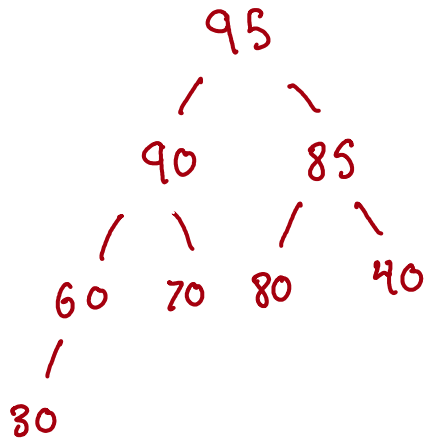
4) Consider the following illustration of a **max-heap** which is does not satisfy the max heap property.



(a) Identify the nodes in the tree which break the max heap property.

95 should not be a child of 70

(b) Draw the resulting valid max-heap after `heapify()` is called on the above heap.

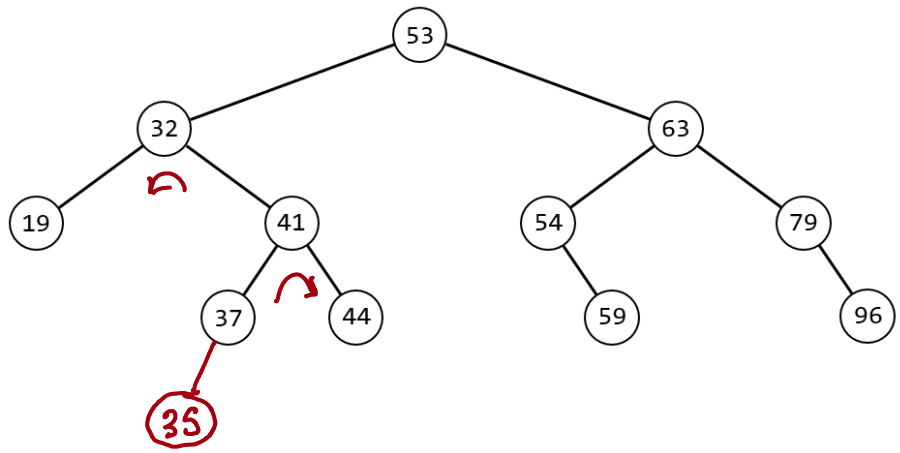


(c) Give the array representation of the max-heap you drew in part (b).

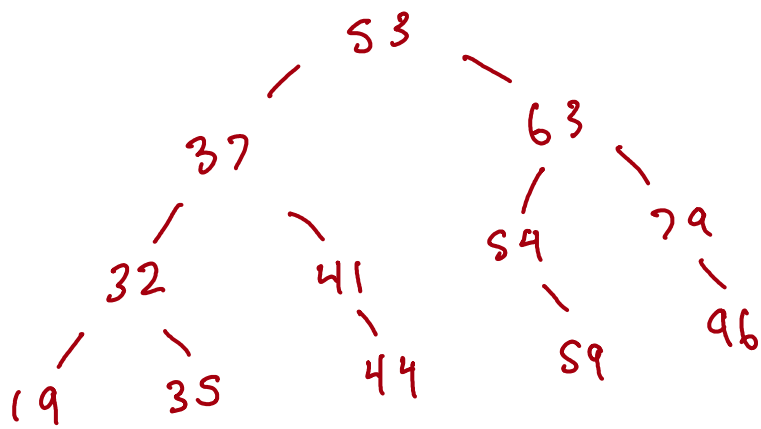
[95, 90, 85, 60, 70, 80, 40, 30]

5) In the following two questions, you are given existing AVL trees and you will be asked about insertions into those AVL trees.

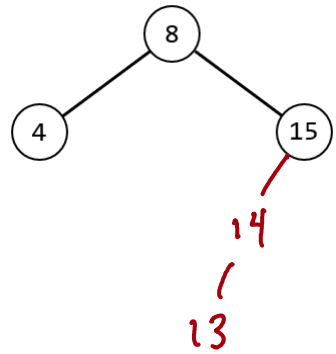
(a) Consider inserting 35 on the following AVL tree. What type of imbalance would occur and what is the root of the unbalanced subtree? Draw the resulting tree after the rotation to rebalance the tree.

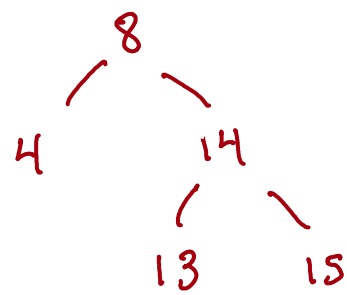


RL imbalance occurs at subtree rooted at 32



(b) Consider the following AVL tree. Give [redacted] an example of the minimum number of insertions which would cause an LL imbalance rooted at the subtree rooted at 15. What rotation(s) is/are done to rebalance the tree? Draw the resulting subtree after you inserted your elements and the tree is rebalanced.



We can insert 14 then 13.

We rebalance by doing a right rotation